

Aula 01 - Reaproveitamento procedural com métodos

Contexto de negócio

Operação diária de agência: duas contas com regras iguais e necessidade de reaproveitamento sem copiar código.

A regra funciona, mas ficou presa no `main`. Como reaproveitar sem copiar contas e ifs em todo lugar?

Tensão operacional

Sem métodos, cada nova conta aumenta repetição de código.

Objetivo técnico da entrega

- Método com parâmetro e retorno.
- Método de exibição.

Recurso técnico desta aula

Recurso inserido: `depositar`, `sacar` e `exibirConta` operando sobre dados recebidos.

Valor prático entregue

- Aumenta velocidade de manutenção ao eliminar duplicação.
- A base fica pronta para evolução controlada da próxima capacidade.

Fluxo operacional explicado

- Preparar os dados de entrada do cenário da aula.
- Executar a regra principal e observar o impacto no estado.
- Confirmar saída de sucesso, erro e borda antes de concluir a etapa.

Código em foco

```
// Classe temporaria de fluxo do programa.
// Neste curso, ela existe para iniciar a execucao em Java.
public class AppGestaoContas {
    static double depositar(double p_saldo, double p_valor) {
        return p_saldo + p_valor;
    }

    static double sacar(double p_saldo, double p_valor) {
        if (p_valor > 0 && p_valor <= p_saldo) {
            return p_saldo - p_valor;
        }
        return p_saldo;
    }
}
```

```
static void exibirConta(String p_titular, double p_saldo) {
    System.out.println("Titular: " + p_titular + " | Saldo: " + p_saldo);
}

public static void main(String[] args) {
    // Aula 01: reaproveitamento procedural com parâmetros e retorno.
    String titularAna = "Ana";
    double saldoAna = 1000.00;

    String titularBruno = "Bruno";
    double saldoBruno = 300.00;

    saldoAna = depositar(saldoAna, 200.00);
    saldoAna = sacar(saldoAna, 150.00);

    saldoBruno = depositar(saldoBruno, 100.00);
    saldoBruno = sacar(saldoBruno, 500.00);

    exibirConta(titularAna, saldoAna);
    exibirConta(titularBruno, saldoBruno);
}
}
```

Leitura técnica orientada

Percorra o código com estes checkpoints de revisão:

- Regra de decisão: mapear condições que aprovam ou negam operações.
- Saída observável: conferir mensagens que comprovam sucesso, erro e bloqueio de regra.

Decisões de projeto das funções

1. Por que `sacar` não tem `else`? Neste caso, não há necessidade técnica de `else` porque o método já retorna no caminho válido e, fora dele, retorna o saldo original no final. A ausência de `else` reduz aninhamento e mantém o fluxo mais direto.
2. `depositar` poderia ter regras? Sim. Exemplo de regras reais:
 - bloquear valor menor ou igual a zero;
 - limitar depósito acima de um teto diário;
 - exigir conta ativa.
3. Assinatura estável: mudar regra sem mudar chamada Se os parâmetros estão bem definidos, podemos evoluir a implementação sem alterar a forma de uso no `main`.

Exemplo de chamada (permanece igual):

```
saldoAna = depositar(saldoAna, 200.00);
```

Exemplo de evolução interna (sem mudar a chamada):

```
static double depositar(double p_saldo, double p_valor) {  
    if (p_valor <= 0) {  
        return p_saldo;  
    }  
    if (p_valor > 10000.00) {  
        return p_saldo;  
    }  
    return p_saldo + p_valor;  
}
```

Discussão de contrato de retorno

Quando o método retorna saldo (**double**), o contrato precisa ficar explícito:

- saldo alterado significa operação aprovada;
- saldo igual ao anterior pode significar operação negada.

Risco de lógica silenciosa:

- se o chamador apenas reutiliza o valor retornado sem validar o resultado, a negativa pode passar despercebida.

Responsabilidade de quem chama:

- comparar **saldoAntes** e **saldoDepois** quando a operação for crítica;
- padronizar mensagem de sucesso e bloqueio no fluxo principal;
- decidir quando vale separar validação (**podeSacar**) de atualização.

Variações para debate técnico:

1. Retornar saldo final (modelo atual): simples e direto para início.
2. Retornar **boolean** e atualizar saldo por referência de estado.
3. Evoluir para resultado explícito com motivo da negativa.

Paradigma em foco: imperativo, procedural e algoritmo

Leitura de paradigma nesta etapa:

- Imperativo: o programa descreve passo a passo o que a máquina deve executar.
- Procedural: organizamos esse passo a passo em funções reutilizáveis.
- Algoritmo: sequência finita de entrada, processamento, decisão e saída para resolver um problema real.

Por que passar valores nas funções no procedural:

- A função não "conhece" a conta por contexto implícito.

- Tudo que a regra precisa deve chegar por parâmetro.
- O contrato da função fica explícito, testável e auditável.

Risco comum quando isso não fica claro:

- chamar função com parâmetro errado, em ordem errada ou incompleto;
- gerar resultado válido na sintaxe, mas incorreto no negócio.

Ponte para a próxima virada de paradigma:

- No procedural, o estado circula entre funções.
- Na POO, estado e comportamento passam a coexistir no objeto.

Perguntas de decisão

1. O método `depositar` sabe quem é Ana ou Bruno?
2. Por que `sacar` pode funcionar sem `else` explícito?
3. Quais regras reais você colocaria em `depositar` sem alterar a assinatura?
4. Por que pensar bem nos parâmetros evita retrabalho de chamadas no futuro?

Variações de cenário

1. Trocar ordem das operações de Bruno.
2. Aplicar dois depósitos seguidos para Ana.

Exercícios progressivos

1. Refatorar `sacar` com `else` explícito e comparar legibilidade, sem alterar assinatura nem chamada.
2. Adicionar regras em `depositar` (valor positivo e teto diário), mantendo a chamada `depositar(saldo, valor)`.

Desafio de equipe

Evoluir o conjunto de funções (`depositar`, `sacar`, `transferencia`) para regras mais realistas sem quebrar contratos de chamada existentes no `main`.

Critérios de aceite dos exercícios

- Regra principal continua válida após alterações.
- Cenário de erro foi testado e tratado sem quebrar execução.
- Código permanece legível e coerente com o nível da aula.
- Assinaturas das funções foram preservadas nas chamadas do `main`.

Fechamento executivo

Reaproveitamos regra, mas os dados ainda estão soltos. Na próxima, vamos aumentar dados por conta e sentir a complexidade procedural crescer.

Revisão rápida (5 minutos)

1. Regra central: qual condição decide aprovação ou bloqueio na aula?

2. Risco real: qual erro operacional essa implementação reduz?
3. Evolução: qual pequena extensão agregaria mais valor sem aumentar acoplamento?
4. As chamadas no `main` permaneceram estáveis após evoluir as regras?

Mini-missão de fechamento (5 min):

- Execute um cenário de borda no Tryit e justifique o resultado em 3 linhas.

Execução no W3Schools Tryit

1. Abrir o compilador online: https://www.w3schools.com/java/tryjava.asp?filename=demo_compiler
2. Colar todo o conteúdo de `AppGestaoContas.java` no editor.
3. Clicar em Run, registrar saída base e depois testar variações e exercícios.
4. Manter uma aba separada para cada exercício e comparar resultados.

Referências W3Schools da aula

- https://www.w3schools.com/java/java_methods.asp
- https://www.w3schools.com/java/java_parameters.asp
- https://www.w3schools.com/java/java_compiler.asp

Leituras complementares

- <https://docs.oracle.com/javase/tutorial/>
- <https://dev.java/learn/>